

MACO: Model Anti-Extraction via Compiler Obfuscation

Yang Li , Minghui Li , Hao Han , *Member, IEEE*, and Liming Fang , *Member, IEEE*

Abstract—Deep neural networks (DNNs) are widely deployed across applications, but growing model sizes and performance demands on edge devices necessitate aggressive compiler-based acceleration, as enabled by frameworks like TVM. Compiling models into standalone executables, however, introduces new security risks, reverse-engineering these binaries can reveal core architectures and enable unauthorized model cloning or tampering. Existing defenses largely rely on trusted execution or encryption, which target weight confidentiality but fail to protect architectural details or operator attributes, often with high runtime costs. To address these gaps, we propose MACO, a multi-tier obfuscation framework built atop TVM, spanning high-level graph rewriting and low-level IR transformations. MACO integrates three transformation categories: operator attribute obfuscation, memory layout perturbation, and dummy branching, covering the full compilation stack. A tiered design enables flexible trade-offs between security and performance. We evaluate MACO on eight common models and three attack types: side-channel, symbolic execution, and deep learning-based extraction. Results show up to 85.8% reduction in attribute inference success and 99% reduction in topology reconstruction, with the lightest tier incurring only $1.036\times$ overhead over the unobfuscated baseline.

Index Terms—Extraction attacks, deep learning, model obfuscation.

I. INTRODUCTION

DEEP Neural Networks (DNNs) have seen rapid adoption across diverse domains, spanning computer vision, natural language processing, and autonomous systems, due to their strong capabilities in pattern recognition and data-driven decision-making. Yet as these models grow in scale and complexity, developers face increasing pressure to optimize inference performance on heterogeneous hardware. This demand has accelerated the emergence of deep learning (DL) compilers, such as TVM [1], which unify high-level model representations into highly optimized, low-level executable code, delivering substantial speedups and reducing cross-platform deployment hurdles. Despite these benefits, the shift to compiler-driven pipelines

also expands the attack surface: the resulting executables encapsulate core architectural details and computation patterns, offering adversaries potential analysis entry points for reverse engineering. Consequently, security concerns arise anew, as motivated attackers may exploit static disassembly or side-channel analysis to clone proprietary networks, posing significant threats to intellectual property and data confidentiality [2].

Recent investigations show that model extraction attacks on compiler-generated binaries have evolved dramatically in both technique and scope. [3] Some adversaries exploit side-channel traces to infer operator (OP) boundaries [4], while others rely on symbolic execution to detect op types in stripped binaries and reconstruct operator sequences from runtime behavior [5], [6]. There are also broader approaches that employ deep learning to identify operator patterns by classifying low-level instruction sequences [7]. These advanced attacks highlight the critical need for robust, compiler-level defenses to safeguard proprietary DNN executables.

Existing defense strategies, such as SecureTVM [8] and NNReArch [9], have yet to address all known risks. For example, SecureTVM focuses on safeguarding model weights with trusted execution environments or selective encryption, but provides limited protection for the model's overall architecture or finer-grained design details. Meanwhile, approaches like NNReArch, which cater to domain-specific accelerator architectures, primarily mitigate side channel vulnerabilities and offer no defense against reverse engineering-based extraction. These limitations and design gaps underscore the need for a more comprehensive, compiler-driven method that obfuscates the model from multiple angles, including architecture, operators, and memory accesses. This vision motivates our work on a multi-faceted obfuscation framework.

A comprehensive defense must first overcome two central obstacles. First, attacks may originate from multiple analysis angles. The obfuscation scheme must cover a wide range of possible entry points. Second, obfuscation inevitably introduces extra overhead, so we require mechanisms to mitigate excessive performance costs. To address these, we propose MACO, a multi-tier compiler obfuscation framework designed to protect deep neural network executable files against static, dynamic, and side-channel reverse-engineering attacks. For the first challenge, MACO distributes obfuscation strategies across different compilation steps and IR layers, influencing every corner of the model. Specifically, our method seamlessly integrates three layers of transformations into TVM's relay and TIR pipelines, including operator attribute modification, memory layout

Received 10 March 2025; revised 2 July 2025; accepted 11 August 2025. Date of publication 21 August 2025; date of current version 4 November 2025. This work was supported in part by the National Key R&D Program of China under Grant 2021YFB3100700, and in part by the National Natural Science Foundation of China under Grant U22B2029, and Grant 62272228. (*Corresponding authors: Hao Han; Liming Fang.*)

The authors are with the College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, Nanjing 210016, China (e-mail: lyang0314@nuaa.edu.cn; leemh@nuaa.edu.cn; hhan@nuaa.edu.cn; fangliming@nuaa.edu.cn).

Digital Object Identifier 10.1109/TDSC.2025.3601219

perturbation, and computational redundancy. Techniques such as dimensional proliferation, kernel enrichment, and bifurcated synthesis expand or split operator attributes to confuse attack detection. Memory access pattern obfuscation and cache alignment perturbation randomize data access to thwart side-channel analysis. Finally, the computational redundancy expansion strategy introduces conditional and dummy branches, complicating both static inspection and dynamic profiling.

For the second challenge, MACO tackles overhead through a tiered architecture that groups different obfuscation strategies into distinct obfuscation levels, allowing users to flexibly tune the depth and breadth of obfuscation based on their cost budget. We design a dual-level intensity control mechanism that maps two global obfuscation parameters onto four progressive tiers (I-IV). Through extensive experiments on representative DNN architectures, we demonstrate that MACO effectively reduces the operator attribute and connectivity recovery rates of multiple attack methods, including symbolic execution, side-channel, and DL-based extraction, while preserving 100% functional correctness. Moreover, a qualitative comparison with state-of-the-art compiler defenses confirms that MACO's layered design and tight integration with the TVM pipeline render it a robust, flexible solution for safeguarding high-value neural networks. Our main contributions are summarized as follows:

- 1) We present the MACO framework, a compiler-level, hardware-agnostic obfuscation system that defends compiled DNNs from extraction attacks. By injecting multi-level transformations across both relay and TIR stages of TVM, MACO comprehensively disrupts static, dynamic, and side-channel analyses. To our knowledge, MACO is the first systems to unify operator attribute distortion, memory layout obfuscation, and computational redundancy within a single end-to-end compilation pipeline, providing broad coverage against adversarial reverse-engineering.
- 2) We propose a dual-tiered obfuscation intensity control mechanism, enabling practitioners to balance security and performance through four incremental levels (Tiers I-IV). This mechanism combines two global parameters so that users can more conveniently customize complex obfuscation transformations, thereby managing overhead in a practical manner. This design allows MACO to accommodate heterogeneous deployment scenarios, from lightweight on-edge systems to high-stakes confidentiality demands, without sacrificing functional correctness.
- 3) We conduct extensive evaluations on eight representative DNN architectures and three advanced attack methodologies, demonstrating the efficacy and overhead profiles of MACO. Our experiments show that MACO can reduce operator attribute recovery by up to 85.8% and connectivity reconstruction by 99% under symbolic execution, side-channel, and DL-based extraction attacks. Meanwhile, we preserve the model's output accuracy (100% consistency) and achieve a minimal overhead of only $1.036\times$ compared to the baseline in Tier I configurations.

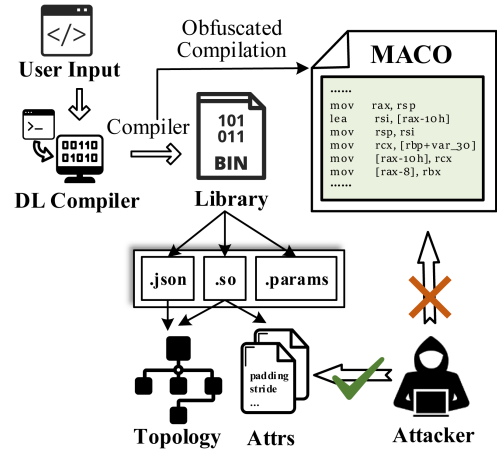


Fig. 1. Reverse extraction attacks in the compilation process.

II. BACKGROUND AND RELATED WORK

This section provides an overview of the deep learning compilation process and existing security concerns related to model extraction attacks, as well as a summary of state-of-the-art model obfuscation strategies.

A. Deep Learning Compilation

DL compilers, exemplified by TVM [1], XLA [10], and MLIR [11], aim to bridge the gap between high-level machine learning frameworks and various backend hardware targets (e.g., CPUs, GPUs, NPUs). Rather than hand-optimizing each operator or relying on vendor-specific libraries, these compilers translate abstract computational graphs into low-level intermediate representations (IR), followed by aggressive transformations such as operator fusion, loop tiling, and memory layout optimizations [12]. In this way, they generate hardware-specific executables capable of achieving near-optimal performance without extensive manual tuning.

Typically, the compilation pipeline is split into multiple IR levels. For instance, TVM employs relay IR and tensor IR (TIR). Relay IR is a high-level graph representation capturing the network architecture (operators, connections) and associated attributes (kernel sizes, channel dimensions, etc.) [13]. TIR is a lower-level loop-centric representation that exposes the fine-grained scheduling and memory access details. Compiling models into a single, self-contained binary or library can significantly simplify cross-platform deployment. However, as these binaries encapsulate key architectural details and scheduling logic, adversaries seeking to reverse-engineer the DNN gain additional attack surfaces [4], [6], as shown in Fig. 1. Once compiled, the code essentially becomes an artifact containing valuable insights into operator shapes, memory layouts, and potential hyperparameters.

B. Model Extraction Attacks

Model extraction attacks aim to replicate or partially reconstruct a proprietary DNN by observing its behavior or analyzing

artifacts such as executables or side-channel signals [14]. Early research on these attacks focused on simple black-box scenarios, where adversaries query the model's input-output pairs to train a 'shadow' network [15]. However, more recent methods delve into the compilation and execution process.

1) *Symbolic Execution & Disassembly*: By disassembling the compiled binary and applying symbolic execution, attackers can track how data flows through loops and conditional branches. DND [5] and BTM [6] rely on partial or selective instrumentation to identify operator boundaries, gather weight shapes, or deduce kernel dimensions. These methods exploit instruction-level semantics and symbolic path constraints to recover operator sequences, even in stripped or statically linked binaries.

2) *Side-Channel Analysis*: Other adversaries collect execution traces such as cache hits/misses [16] or performance counters [17], enabling them to infer operator boundaries or compute intensities. Hu et al. [4] proposed a model extraction framework that reverse-engineers DNN architectures by establishing empirical connections between operational traces (e.g., read/write magnitudes extracted from memory side-channels) and the latent architectural properties of target DNN models. Such attacks leverage fine-grained temporal or spatial access patterns to correlate hardware-level leakage with high-level operator behavior, often without needing code visibility.

3) *Learning-Based Reconstructions*: Recent studies have explored DL-based model extraction attacks, learning the contextual relationship of operators to infer DNN architectures and parameters. Li et al. [7] combines code representation learning method and binary analysis to enhance reverse engineering accuracy against compiled deep learning models.

C. Model Obfuscation

Model obfuscation is a critical technique to defend against neural network architecture reverse engineering, ensuring the confidentiality of model structures, hyperparameters, and computational graphs [18], [19]. Existing works have explored different strategies to mitigate such threats. Luo et al. [9] proposed the framework through introducing scheduling transformations that alter the execution order and memory access patterns of tensor computations without compromising model correctness. Huang et al. [8] introduced the method that allows model providers to selectively configure which layers to protect. During the compilation phase, the model provider can specify particular layers that require protection. SecureTVM maps the computations of these protected layers to a secure execution domain, while non-protected layers continue to run in the standard domain. However, selective protection means that certain parts of the model might still be vulnerable to reverse engineering attacks if not properly secured. The trade-off between privacy and computational efficiency remains a challenge.

Beyond compiler-based defenses, prior works have explored model watermarking [20], pruning-based obfuscation [21], and encryption-based protections [22]. These methods typically focus on ownership verification, model size reduction, or parameter confidentiality, can not resist the novel attack mentioned in the

previous section. In contrast, MACO targets structural obfuscation at the IR level, aiming to disrupt architecture-level reverse engineering while preserving full inference functionality.

III. MACO'S DESIGN

In this section, we introduce the design principles of MACO. The overall architecture is divided into three modules: the operator layer, memory layer, and graph layer. Each designed to obfuscate a different attacker perspective. Fig. 2 illustrates an overview of the MACO architecture. The upper portion depicts the normal compilation flow, while the lower portion highlights MACO's obfuscation pathway. At the two principal IR stages, MACO inserts different obfuscation strategies, which are detailed in the following sections. However, before delving into these strategies, we first articulate the threat model that MACO is designed to defend against.

A. Threat Model

We assume an adversary who has full access to the compiled binary (TVM) of a deep learning model, including the ability to disassemble the executable, analyze low-level intermediate representations (e.g., LLVM IR or assembly), and apply symbolic execution or static data-flow analysis. The attacker is also assumed to have profiling capabilities, such as access to performance counters or cache behavior via side-channel monitoring. However, we do not assume access to the original model source code (e.g., PyTorch or TensorFlow definitions), training data, or runtime input/output pairs. The adversary's goal is to reconstruct the model's architecture, operator attributes, or computational topology based on binary-level signals. Query-Based extraction techniques (e.g., API probing or input-output imitation) are out of scope.

B. Operator Attributes Obfuscation Set

For reverse extraction attacks, a crucial step in reconstructing the model is inferring or computing operator attributes. To impede this process, we introduce mathematically equivalent redundancies into operators. Specifically, we define six obfuscation techniques within the obfuscation set.

1) *Dimensional Proliferation (DP)*: It enhances model security by expanding the dimensionality of the weight tensor, introducing controlled redundancy without altering the output of the operator. This transformation complicates reverse-engineering efforts by modifying the internal model structure, increasing the number of output channels while preserving functional equivalence. Consider a convolutional operator with weight tensor $W \in \mathbb{R}^{k_1 \times k_2 \times c \times j}$, where k_1 and k_2 are the kernel dimensions, c is the number of input channels, and j is the number of output channels. The output for a given input x is computed as $y = W * x$, where $*$ denotes the convolution operation.

In DP, we expand the weight tensor to $W' \in \mathbb{R}^{k_1 \times k_2 \times c \times j'}$, with $j' = \alpha j$ and $\alpha > 1$, introducing redundancy by increasing the number of output channels. The new weight tensor produces an output $y' = W' * x$, where the additional output channels are

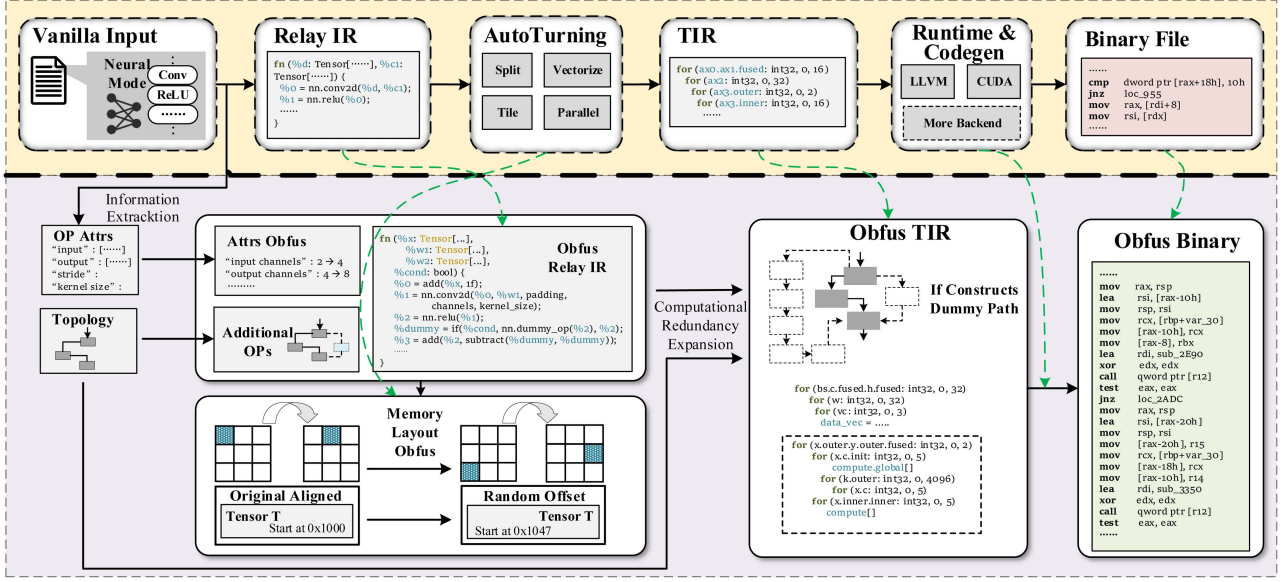


Fig. 2. The framework of MACO. Providing compilation obfuscation at different compilation stages.

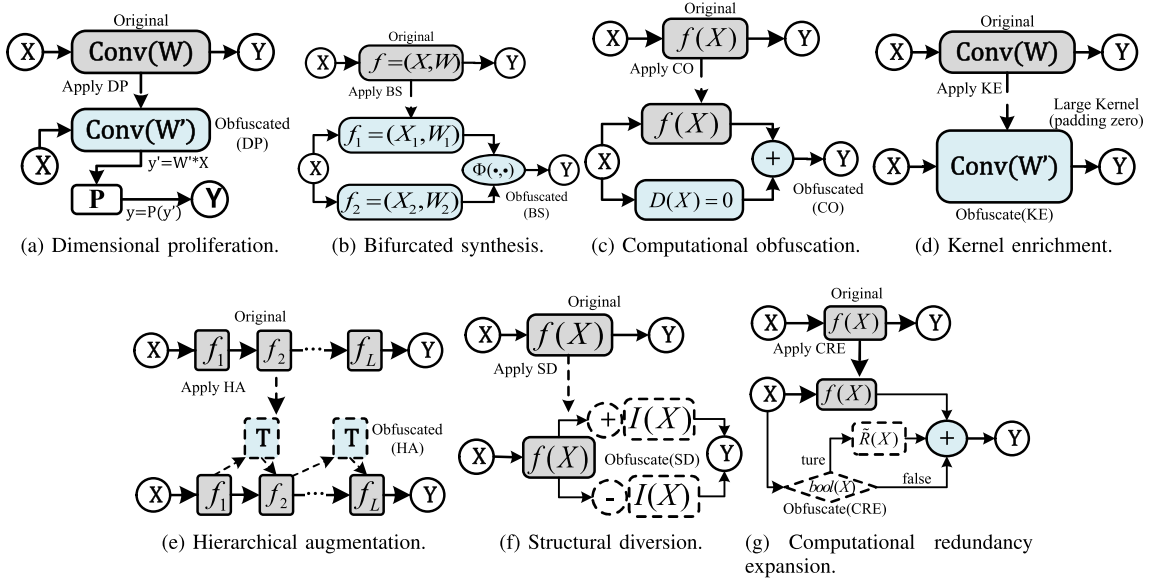


Fig. 3. Operator attributes obfuscation strategy set.

not necessary for the original functionality but alter the internal structure of the computation.

To preserve the original output, we introduce a projection operator P that maps the expanded output back to the original number of channels:

$$y = P(W' * x) \quad (1)$$

where P ensures that only the first j channels of the expanded output are used. A typical form of P is the identity matrix for the first j rows, followed by zero rows for the remaining channels:

$$P = \begin{bmatrix} I_j \\ 0 \end{bmatrix} \quad (2)$$

We illustrate the principle more clearly in Fig. 3(a). This projection guarantees that the final output is mathematically identical to the output of the original operator, while introducing additional complexity to the model structure. At the same time, by introducing redundant channels, the memory access patterns will also be altered, thereby interfering with reverse analysis.

2) *Bifurcated Synthesis (BS)*: It decomposes a neural network operator into multiple sub-operations, which are executed either sequentially or in parallel. This will complicate the tracking of the computational flow. For example, consider an original operator defined by a function $f(X, W): \mathbb{R}^n \rightarrow \mathbb{R}^m$, where X is the input tensor and W is the weight tensor. In this transformation, both the input and weight tensors are partitioned

along a chosen dimension. The input tensor X and weight tensor W are split into two complementary parts:

$$X = \text{concat}(X_1, X_2), \quad W = \text{concat}(W_1, W_2) \quad (3)$$

where the split ratio is controlled by a parameter $\beta \in (0, 1)$. The function f is then decomposed into two sub-functions:

$$f_1(X_1, W_1) \quad \text{and} \quad f_2(X_2, W_2) \quad (4)$$

which are computed independently. These two sub-functions are later combined using a recombination operator Φ , such as concatenation or element-wise summation, to yield the original output:

$$\begin{aligned} f(X, W) &= f(\text{concat}(X_1, X_2), \text{concat}(W_1, W_2)) \\ &\equiv \Phi(f_1(X_1, W_1), f_2(X_2, W_2)) \end{aligned} \quad (5)$$

Fig. 3(b) illustrates the principle. This transformation obfuscates the model's execution trace, making it harder for attackers to reconstruct the original sequence of operations. However, it does so without affecting the functional output, maintaining the operator's behavior while introducing additional computational paths.

3) *Computational Obfuscation (CO)*: It introduces redundant yet numerically neutral operations into the network, such as identity multiplications, zero additions, or padding. This additional computation increases the number of tensor operations, thereby obscuring performance traces while leaving the final output unchanged. For example, we have an original operation defined as $Y = f(X)$, where f is an operator function and X is the input. In the CO transformation, a redundant term $D(X)$ is incorporated so that the output becomes $Y' = f(X) + D(X)$. Here, $D(X)$ is designed to be neutral (e.g., `zeros_like(X)`) or an identity mapping $I_n X I_m^T$, with I_n and I_m as appropriately sized identity matrices), ensuring that $Y' = f(X)$ in effect, as shown in Fig. 3(c). Although these redundant operations do not alter the functional output, they modify the computational graph and memory access patterns, thus complicating reverse-engineering efforts.

4) *Hierarchical Augmentation (HA)*: It increases the effective depth of a neural network by inserting identity transformations between existing operators. While these inserted operations do not change the output (i.e., they are no-op transformations), they introduce additional computational steps that obscure the operator sequence, without altering its inference behavior. Let the original network be expressed as a composition of operator functions:

$$F(X) = f_L \circ f_{L-1} \circ \dots \circ f_1(X) \quad (6)$$

where f_i represents an operator transformation and X is the input tensor. In this transformation, we introduce identity transformations T between consecutive operators. This transforms the network into:

$$F'(X) = f_L \circ T \circ f_{L-1} \circ T \circ \dots \circ T \circ f_1(X) \quad (7)$$

where $T(x)$ is an identity mapping, i.e., $T(x) = x$ for all x . A typical instantiation of T is the identity convolution, defined as

$T(x) = I_{1 \times 1} * x$, where $I_{1 \times 1}$ is a 1×1 convolution kernel with weights set to the identity matrix. This guarantees that $F'(X) = F(X)$, while the augmented computation graph now contains additional operators that obscure the original structure, as shown in Fig. 3(e). These identity operations do not affect the output but add unnecessary complexity to the computation flow.

5) *Structural Diversion (SD)*: It modifies the network architecture by inserting redundant operations that do not affect the final output, but change the runtime execution characteristics and memory access patterns. Unlike other obfuscation methods that focus on deepening or bifurcating the network, Structural Diversion introduces ineffective operators that obscure the computational flow. We give an original operator function $f(X) : \mathbb{R}^n \rightarrow \mathbb{R}^m$, where X is the input tensor. To implement SD, we introduce an identity mapping I such that $I(X) = X$, and construct a redundant operation by adding and subtracting the same quantity, which effectively has no impact on the output. This operation is expressed as:

$$Y = f(X) + (I(X) - I(X)) \quad (8)$$

Fig. 3(f) illustrates the principle. Although this may appear straightforward, the inclusion of redundant addition and subtraction operations alters the memory access patterns and computational traces, thereby strengthening the system's resilience to reverse engineering efforts.

6) *Kernel Enrichment (KE)*: The KE modifies convolutional layers by expanding the spatial dimensions of the kernel, ensuring the output remains functionally equivalent to the original layer. This transformation involves embedding the original kernel weights within a larger kernel, padding the additional positions with zeros. We illustrate this with a convolution operator as an example $Y = \text{conv}(X, W)$, where $W \in \mathbb{R}^{k_1 \times k_2 \times c \times j}$ represents the original kernel weights, X is the input tensor, and Y is the output tensor. In the KE transformation, we expand the kernel dimensions by adding rows and columns. Let the new dimensions be:

$$k'_1 = k_1 + \delta_1, \quad k'_2 = k_2 + \delta_2 \quad (9)$$

where δ_1 and δ_2 represent the number of additional rows and columns, respectively. This creates a new kernel $W' \in \mathbb{R}^{k'_1 \times k'_2 \times c \times j}$. We define the kernel W' as follows:

$$W'(i, j, :, :) = W(i - \lfloor \delta_1/2 \rfloor, j - \lfloor \delta_2/2 \rfloor, :, :) \quad (10)$$

where the kernel elements from the original kernel W are embedded within the enlarged kernel W' . The extra positions in the kernel are padded with zeros.

As shown in Fig. 3(d), the input tensor X is appropriately padded to match the new kernel dimensions, and the convolution operation is mathematically equivalent to the original operation. If the indices i and j fall within the original kernel's dimensions (i.e., within the bounds set by δ_1 and δ_2), the corresponding elements of W' are taken from the original kernel W . Otherwise, the additional positions in W' are padded with zeros. This transformation ensures that the convolution with the enlarged kernel W' produces the same result as with the original kernel W , while introducing changes in computational behavior.

C. Tensor-Level Memory Layout Obfuscation

Beyond operator attribute modifications, we introduce tensor layout transformations to perturb memory access patterns. Attackers relying on side-channel signals, such as cache hits and DRAM transactions, will be misled by our dynamic memory perturbations.

1) *Memory Access Pattern Obfuscation (MAPO)*: The MAPO module aims to obfuscate the memory access patterns by manipulating the order in which tensor data is accessed. By reordering tensor accesses or introducing random access patterns, the model's memory layout becomes more difficult to predict.

In a typical tensor operation, data is accessed in a sequential or predictable manner, which could allow attackers to infer the layout of the tensor in memory. The key idea behind MAPO is to dynamically alter the order in which tensor elements are accessed during computation. This can be achieved by permuting the indices used to access the tensor or introducing random access patterns based on a predefined or dynamic scheme.

For instance, consider a tensor T with shape $\mathbb{R}^{n_1 \times n_2 \times \dots \times n_k}$, where $n_1, n_2, \dots, n_k$ are the dimensions of the tensor. Instead of accessing the elements of T in a straightforward sequence, we introduce a permutation P that randomizes the access order by applying a permutation function to the tensor indices. Specifically, the access pattern is altered at runtime such that the tensor elements are accessed in a randomized order, effectively making the original access sequence unpredictable. This perturbation in the access order ensures that the tensor's data is scattered across memory in a way that prevents attackers from inferring the tensor layout based on predictable memory access patterns. This transformation still does not affect the model's output, but provides an additional obfuscation, making attempts to reverse-engineer the memory organization through side-channel analysis (such as cache profiling or DRAM transaction tracking) more complicated.

2) *Cache and Alignment Perturbation (CAP)*: The CAP module introduces variability in memory access patterns by altering memory alignment or cache utilization strategies. By adjusting how data is aligned in memory or how it is accessed from the cache, CAP makes it significantly more difficult for attackers to infer the data structure by analyzing cache hits and misses.

During the compilation and inference execution process, data is often aligned in memory according to a fixed scheme that optimizes cache efficiency. Attackers can exploit this predictable alignment to infer the structure of tensors. If the variability in alignment can be introduced, it would effectively make the memory access patterns more difficult to trace.

One way to achieve this is by altering the memory alignment of the tensors during allocation. Instead of aligning the tensor data at a fixed boundary, we introduce a random alignment factor α , which serves as a random offset applied to the starting memory address of tensor T . This random offset causes the tensor to be placed at non-standard memory locations, disrupting potential analysis based on fixed memory alignment. We can dynamically adjust the alignment factor α at runtime. This effectively reduces the predictability of memory access patterns

and increases the difficulty of inferring the underlying memory structure of the inference model.

D. Computational Redundancy Expansion (CRE)

The CRE technique deliberately embeds redundant computations into the neural network's execution graph without altering its final outputs, thereby complicating reverse-engineering efforts. CRE achieves this via two complementary strategies. First, from the overall graph structure perspective, fake control flows are introduced at the relay IR level using conditional branches (e.g., if constructs). In this approach, redundant neural network computations are conditionally activated based on runtime parameters. For example, consider a primary computation $f(X)$ for input X . A redundant computation $R(X)$ is conditionally executed according to a runtime-dependent condition $C(X)$, and the augmented computation is defined as

$$f'(X) = f(X) + \text{if}(C(X), \tilde{R}(X), 0) \quad (11)$$

where $\tilde{R}(X)$ is a mathematically neutral operation (e.g., a sequence of arithmetic operations that cancel out) ensuring that $f'(X)$ remains equivalent to $f(X)$. We illustrate the principle more clearly in Fig. 3(g). Although these conditional branches are executed at runtime (and thus affect scheduling and memory access patterns), their results are designed to cancel out and do not feed back into the main computation. Second, at the compilation level, particularly during the TIR lowering process, dummy operators and dedicated dummy paths are inserted into the computation graph. These dummy paths are implemented as true dummy operators whose outputs are mapped or scaled to an insignificantly small range before being merged back into the main path. Moreover, dynamic indexing is introduced within these dummy paths through runtime-dependent index computations (such as random or unrelated index calculations). This dynamic indexing makes the memory access patterns in the dummy paths non-deterministic, thereby disrupting static analysis of path dependencies. Despite being computed during inference, the output of these dummy paths is deliberately designed to have no effect on the final result.

In summary, CRE embeds both conditional and dummy operations into the execution graph. These redundant computations are conditionally activated and dynamically indexed so that they perturb scheduling, memory access, and static analysis, while their mathematical contributions cancel out, ensuring that the main computational output remains unchanged.

Collectively, the three obfuscation transformations, derived from different levels and perspectives, are seamlessly integrated into the TVM compilation pipeline, ensuring that the obfuscated model retains its original accuracy while posing significant challenges for model extraction and reverse-engineering attacks. In the subsequent section, we detail the implementation strategies that realize these transformations within TVM.

IV. IMPLEMENTATION AND EXECUTION FLOW OF MACO

We implement MACO as a modular transformation framework within TVM, leveraging its relay and TIR layers to systematically inject obfuscation throughout the compilation

```

Original TIR:
for (n = 0; n < N; n++)
  for (oc = 0; oc < OC; oc++)
    for (h = 0; h < H; h++)
      for (w = 0; w < W; w++)
        output[n][oc][h][w] = 0;
        for (ic = 0; ic < IC; ic++)
          for (kh = 0; kh < KH; kh++)
            for (kw = 0; kw < KW; kw++)
              output[n][oc][h][w] +=
                data[n][ic][h+kh][w+kw] * weight[oc][ic][kh][kw];
Obfuscated TIR with dimensional proliferation:
for (n = 0; n < N; n++)
  // Compute augmented output with redundant channels (OC * alpha)
  for (oc = 0; oc < OC * alpha; oc++)
    for (h = 0; h < H; h++)
      for (w = 0; w < W; w++)
        temp_output[n][oc][h][w] = 0;
        for (ic = 0; ic < IC; ic++)
          for (kh = 0; kh < KH; kh++)
            for (kw = 0; kw < KW; kw++)
              temp_output[n][oc][h][w] +=
                data[n][ic][h+kh][w+kw] * weight_augmented[oc][ic][kh][kw];
  // Apply projection to recover original channel count
  for (oc = 0; oc < OC; oc++)
    for (h = 0; h < H; h++)
      for (w = 0; w < W; w++)
        output[n][oc][h][w] = temp_output[n][oc][h][w] * mask[oc];

```

Fig. 4. An example of DP obfuscation for 2D convolution in TIR form. DP transformation has extended the loop over output channels by introducing redundant channels and then applying a projection to recover the original shape.

process. The end-to-end execution flow begins with model import and high-level relay rewriting, proceeds through metadata propagation and operator lowering, and culminates in backend code generation with embedded obfuscation logic. At the operator level, attribute obfuscation techniques, such as DP, KE, BS, and HA, are implemented through a combination of relay rewriting, TE-level operator extension, and low-level schedule manipulation. These transformations jointly produce semantically equivalent yet structurally diversified operators, and are designed to persist through TIR lowering and backend code generation. For example, DP increases the output channel dimensionality by appending redundant filters that lie in the null space of a projection operator, followed by an explicit projection that recovers the original output shape. KE enlarges the convolution kernel by symmetric zero-padding and adjusts the input accordingly to preserve output equivalence. These transformations modify the operator's loop structures and memory access patterns, as illustrated in Figs. 4 and 5. Due to space constraints, we illustrate only the TIR-level structural differences resulting from DP and KE. The full operator attribute obfuscation pipeline and its integration into the compilation process are presented in Algorithm 1. At the memory level, layout perturbation is achieved through two complementary mechanisms. MAPO introduces index permutations by altering loop access patterns, and CAP shifts buffer base addresses by runtime-determined offsets. These memory-level changes are coordinated through the memory planning phase and concretely injected into the TIR via dynamic indexing logic, bound checks, and modified allocation calls. Finally, dummy branching obfuscation is implemented through CRE, which injects runtime-dependent control flow and data paths at both IR stages. Specifically, Relay passes insert conditional subgraphs with neutral arithmetic, while TIR-level modifications introduce dummy operators using randomized or input-derived index expressions, ensuring execution traces diverge across runs while leaving output unchanged. Together,

```

Original TIR:
for (n = 0; n < N; n++)
  for (c = 0; c < C; c++)
    for (h = 0; h < H; h++)
      for (w = 0; w < W; w++)
        output[n][c][h][w] = conv(input[n], weight[c]);
Obfuscated TIR with kernel enrichment:
for (n = 0; n < N; n++)
  for (c = 0; c < C; c++)
    for (h = 0; h < H; h++)
      for (w = 0; w < W; w++)
        temp = 0;
        for (kh = 0; kh < k_h + 2*p_h; kh++)
          for (kw = 0; kw < k_w + 2*p_w; kw++)
            if (kh >= p_h && kh < k_h + p_h &&
                kw >= p_w && kw < k_w + p_w)
              temp += input[n][...offset...] *
                weight_enriched[c][kh - p_h][kw - p_w];
            else
              temp += 0; // Zero padding region
        output[n][c][h][w] = temp;

```

Fig. 5. KE obfuscation for 2D convolution in TIR.

these tightly integrated components constitute the core implementation of MACO, delivering a coherent execution flow that propagates structural obfuscation from high-level graph transformations down to low-level memory access patterns, while remaining compatible with TVM's optimization and code generation pipeline.

V. OBFUSCATION INTENSITY CONTROL STRATEGY

To balance security gains against computational overhead, our framework employs a dual-level control mechanism that regulates both the frequency and the combination of obfuscation strategies applied to a deep learning computation graph. Rather than uniformly modifying all eligible operators with every available technique, our mechanism makes two key decisions. (1) For each obfuscation strategy, determine the fraction of eligible operators to be transformed. (2) Based on a user-selected obfuscation tier, choose which subsets of strategies to apply.

A global intensity parameter, denoted as $\kappa \in [0, 1]$, governs the degree of transformation for each strategy. For every obfuscation technique i (e.g., DP, BS, MAPO, CAP, CO, HA, and SD), we define a selection probability function $p_i(\kappa)$ that determines the proportion of eligible operators to be modified. Additionally, we design a scaling function $f_i(\kappa)$ to adjust strategy-specific parameters (such as the expansion factor in DP). For example, for DP, the number of output channels under the control of κ is computed as

$$j' = j + \lfloor (\alpha - 1) \cdot \kappa \cdot j \rfloor \quad (12)$$

thereby allowing the modification strength to scale with κ .

In addition, we introduce a second global parameter, η , which is used exclusively for controlling the CRE strategy. Given that CRE is the most computationally intensive method, it is applied only at the highest obfuscation tier. Rather than adjusting the magnitude of each dummy operation, which remains fixed at a very small constant $\epsilon \ll 1$ to ensure neutrality, η specifies the fraction of eligible nodes at which a redundant branch is injected. For instance, if there are N eligible nodes for CRE, approximately $\eta \times N$ extra dummy operators will be added to

Algorithm 1: Unified Obfuscation Transformation of Operator Attributes.

Input: Original graph G , obfuscation parameters:
 DP: expansion factor $\alpha > 1$;
 BS: partition ratio $\beta \in (0, 1)$;
 CO: redundant operation function $\tilde{D}(\cdot)$ (e.g., an identity multiplication or zero-generation function);
 HA: identity transformation T ;
 SD: redundant branch insertion;
 KE: kernel padding parameters δ_1, δ_2 .

Output: Obfuscated graph G' with preserved functional output.

```

1: // Graph Transformation Phase
2: for each operator node  $n$  in  $G$  do
3:   if  $n$  is a convolution or fully-connected operator then
4:     if DP is enabled then
5:       Expand weight tensor:  $W' \leftarrow [W, \Delta W]$ , where  $\Delta W$  satisfies  $P \Delta W = 0$ , and the new output channels are  $j' = j + \lfloor (\alpha - 1) \cdot j \rfloor$ ;
6:       Replace  $n$  with an augmented operator that computes  $y' = W' * x$ , and insert a projection operator  $P$  to recover  $y = P(y')$ ;
7:     end if
8:     if KE is enabled and  $n$  is convolutional then
9:       Compute enriched kernel dimensions:  $k'_1 = k_1 + \delta_1, k'_2 = k_2 + \delta_2$ .
10:      Replace  $W$  with  $W'$ , embedding  $W$  at the center and padding extra elements with zeros.
11:      Adjust input  $x$  via appropriate padding to maintain  $\text{conv}(x, W') = \text{conv}(x, W)$ .
12:    end if
13:  end if
14:  if  $n$  is eligible for BS then
15:    Partition input  $x$  into  $x_1$  and  $x_2$  and weight  $W$  into  $W_1$  and  $W_2$  based on  $\beta$ .
16:    Replace  $n$  with two branch operators computing  $y_1 = f_1(x_1, W_1)$  and  $y_2 = f_2(x_2, W_2)$ .
17:    Insert a recombination operator  $\Phi$  such that  $y = \Phi(y_1, y_2)$ .
18:  end if
19:  if  $n$  is selected for CO then
20:    Modify  $n$  to compute  $y = f(x) + (D(x) - D(x))$ 
    ▷The redundant term  $\tilde{D}(x)$  is designed to be numerically neutral.
21:  end if
22:  if HA is enabled at layer boundaries then
23:    Insert an identity transformation  $T$  (e.g., a  $1 \times 1$  convolution or frozen batch norm) between adjacent operators.
24:  end if
25:  if SD is enabled then
26:    Inject a redundant branch into  $n$  by computing  $y = f(x) + (I(x) - I(x))$ ,
27:    where  $I(x)$  is an identity mapping; this branch perturbs memory access without affecting  $y$ .
28:  end if

```

```

29: end for
30: // Operator Extension and Annotation Phase
31: for each transformed operator in  $G'$  do
32:   Annotate with metadata (e.g., fusion hints, memory layout adjustments) to guide low-level optimizations.
33: end for
34: // Low-Level Code Generation and Scheduling Phase
35: Invoke TVM's code generation backend to translate the augmented Relay IR into optimized C++/LLVM kernels.
36: Apply specialized scheduling primitives to
37: Fuse redundant operations (e.g., masking, concatenation) to minimize overhead.
38: Leverage vectorized instructions and cache-aware memory management.
39: Preserve dynamic conditional execution paths to ensure the dummy and redundant computations are maintained at runtime.
40: return the obfuscated graph  $G'$ .

```

the graph. This provides a direct knob to control the overall additional operator count. We further partition the overall obfuscation configuration into four distinct tiers:

- *Tier I: Conservative Obfuscation.* Only low-overhead strategies, such as MAPO and CAP, are enabled.
- *Tier II: Moderate Obfuscation.* In addition to Tier I, modest-cost strategies like DP and BS are applied.
- *Tier III: Enhanced Obfuscation.* Tier II is augmented with identity-based obfuscation techniques such as CO and HA.
- *Tier IV: Comprehensive Obfuscation.* All strategies, including the computationally heavy CRE, are activated.

The tier selection is implemented by defining a mapping from the desired obfuscation level to a subset of strategies. At the implementation process, both κ and η are embedded as metadata within the relay IR. During the IR transformation phase, each obfuscation pass consults these parameters. A strategy is applied to an eligible operator with probability $p_i(\kappa)$, and its configuration is scaled by $f_i(\kappa)$. For CRE, the overall number of dummy branches inserted is controlled by η , ensuring that the additional operators comprise a predetermined fraction of the eligible set. These modifications propagate through to the TIR generation phase, where dynamic index adjustments and modified loop bounds reflect the intended obfuscation intensity. This two-level control mechanism thus provides a systematic and configurable trade-off between security and performance.

Execution Flow of Obfuscation Intensity Control: Algorithm 2 outlines our unified strategy for selecting and applying obfuscation transformations based on the global parameters κ and η , and on the chosen obfuscation tier. Note that the final output of the transformation pipeline is a fully obfuscated TIR module M' , which incorporates both high-level relay modifications and low-level scheduling perturbations. For better understanding, we also illustrate this process in Fig. 6.

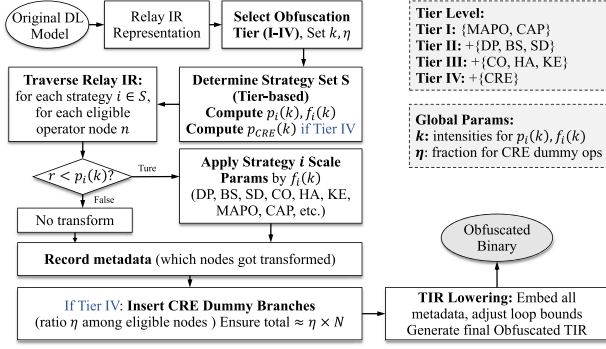


Fig. 6. Detailed obfuscation intensity control flow. Starting from the original DL model, we obtain a relay IR in TVM. We then choose an obfuscation tier (I-IV) and set the global parameters κ and η . According to the tier, a subset of strategies S is determined, and for each strategy $i \in S$, each eligible operator is transformed with probability $p_i(\kappa)$, scaling parameters by $f_i(\kappa)$. Tier IV additionally injects CRE-based dummy branches (η controls the fraction of nodes). All metadata and transformations are embedded in TIR lowering, producing the final obfuscated binary.

VI. EVALUATION

Our goal is to comprehensively assess the security effectiveness, performance overhead, functional correctness, and advancement of MACO in defending against model extraction and reverse-engineering attacks. In Section VI-B, we describe the experimental setup, including datasets, model architectures, metrics, and baselines. Then, Sections VI-C, VI-D, VI-E, and VI-F detail our findings for each research question.

A. Environments

Our MACO framework is primarily implemented in C++ and Python, integrating directly with the TVM compilation pipeline. The current prototype spans approximately 5.2 K lines of code (LOC), including custom relay passes for operator transformations, TIR-level obfuscation logic, and auxiliary scripts for compiler configuration and metadata management.

In the experiments reported below, we target 64-bit ELF executable files generated by TVM, focusing on the x86-64 architecture. We tested MACO on a 64-bit Ubuntu 20.04 system, running on an AMD Ryzen 7 5800X at 3.80 GHz with 32 GB of memory, paired with an NVIDIA GeForce RTX 3090 for GPU-based trials. We used TVMv0.8, compiled with LLVM11 as the backend.

For the disassembly and low-level binary inspection required by reverse extraction attacks, we employed IDA Pro 7.5 in combination with IDAPython scripting to analyze how our obfuscated binaries appear under static analysis. Additionally, to accommodate specific side-channel attack configurations, we incorporated performance profiling and custom logging hooks within the system runtime to capture side-channel traces across different obfuscation tiers.

B. Experiments Setup

1) *Datasets and Model Architectures:* For the evaluation of MACO, we employ a diverse set of deep learning models primarily sourced from the *ONNX Model Zoo* [23].

Algorithm 2: Obfuscation Intensity and Strategy Selection.

Input: Original relay IR graph G ;

Global intensity parameter $\kappa \in [0, 1]$;

Global CRE operator ratio $\eta \in [0, 1]$;

The count of all operators N .

Predefined obfuscation tiers with strategy sets:

Tier I: {MAPO, CAP};

Tier II: Tier I $\cup$ {DP, BS, SD};

Tier III: Tier II $\cup$ {CO, HA, KE};

Tier IV: Tier III $\cup$ {CRE};

For each strategy i , a selection probability function

$p_i(\kappa)$ and scaling function $f_i(\kappa)$;

Output: Fully obfuscated TIR module M' that reflects all applied transformations.

1: // **Tier Selection:** Based on choice, determine the set S of obfuscation strategies to be applied.

2: **for** each strategy $i \in S$ **do**

3: **for** each eligible operator node $n \in G$ for strategy i **do**

4: Draw a random number $r \sim U(0, 1)$;

5: **if** $r < p_i(\kappa)$ **then**

6: Apply transformation i on node n with parameters scaled by $f_i(\kappa)$;

7: **end if**

8: **end for**

9: **end for**

10: **if** CRE is enabled in Tier IV **then**

11: **for** each eligible operator node n for CRE **do**

12: With probability $p_{\text{CRE}}(\kappa)$, insert a dummy branch at node n ;

13: **end for**

14: Ensure that the total number of inserted dummy operators is approximately $\eta \times N$;

15: **end if**

16: Embed all chosen parameters and selection outcomes as metadata in G .

17: Propagate the metadata through TIR lowering to generate the final TIR module M' that incorporates dynamic index adjustments and modified loop bounds.

18: **return** M' .

These pretrained models already provide complete parameters and fully functional inference capabilities. Specifically, we select VGG16 [24] and ResNet18/34/50 [25] as strong baselines to assess MACO's capacity to handle both shallow and moderately deep architectures. Additionally, we include DenseNet121 [26] and Xception [27] to introduce more complex architectures with intricate skip connections or depthwise separable convolutions, thereby posing non-trivial challenges for obfuscation. Finally, we deliberately add MobileNetV2 [28] and SqueezeNet [29], lightweight, optimized networks often deployed on resource-constrained devices, to highlight MACO's applicability to models demanding efficiency in both size and speed.

These models span a broad range of convolutional network designs, from relatively simple macro-architectures (like VGG16) to modern, highly optimized or deeper backbones (like

Xception). They encompass various channel structures, skip-connection patterns, and convolution variants. By evaluating MACO's obfuscation strategies on *both standard and advanced* CNNs, we gain insight into MACO's robustness and generality across different complexity levels.

In line with common use cases for ONNX-based deployments, we compile each model into binary file through TVM's pipeline. We apply MACO transformations at both the relay and TIR levels, examining metrics such as obfuscation strength, runtime overhead, and retained accuracy in subsequent experiments.

2) *Metrics*: We use both security and system performance metrics.

Security Metrics: The evaluation of security in our approach is based on three key metrics. Normalized graph edit distance (nGED) is used to indicate the obfuscation effect on high-level graph structures. Operator attribute recovery accuracy (OARA) quantifies the percentage of operator attributes (e.g., kernel size, number of output channels) correctly identified by the attacker compared to the ground truth from the original model. An attribute is counted as correctly recovered when all of its associated fields exactly match the original operator. Operator connectivity accuracy (OCA) assesses the percentage of topological connections that the attacker correctly reconstructs. A connection is considered correct only if the source and target operators, along with their edge direction, precisely match the original dataflow graph, partial matches or misaligned branches are treated as errors.

System Performance Metrics: The performance evaluation of our system is based on three primary metrics. Latency (ms) measures the time required to execute a single inference (batch size = 1) after compilation with TVM at optimization level -O0, reflecting the raw computational cost. Memory Overhead quantifies the relative increase in peak memory usage compared to an unobfuscated (vanilla) baseline. Output consistency represents the percentage of inference instances where the obfuscated model produces the identical output label as the original model.

In addition, we enable a suite of transformations (DP, BS, CO, HA, SD, KE, MAPO, CAP, and CRE) at the relay and TIR levels based on the tier level, as described in Section III of the paper.

3) *Baseline*: We evaluate MACO against three representative attack methodologies drawn from prior works, BTM [6], DS [4], and REDLC [7], which collectively span symbolic execution-based, side-channel, and DL-driven model extraction strategies. In addition, these methods target different hardware backends. BTM and REDLC are executed on CPU platforms, whereas DS operates on GPUs. To further compare MACO's effectiveness with domain-leading defenses, we also include two SOTA compiler-based protection frameworks in our qualitative baseline. NNReArch [9] is a tensor scheduling solution against reverse-engineering. And SecureTVM [8] is a selective privacy-preserving compiler for TVM. We choose these systems as they represent cutting-edge approaches to safeguarding neural networks at the compilation level.

C. Effectiveness Against Model Extraction Attack

1) *Measuring Graph Topology Divergence*: We first design a controlled experiment to quantitatively assess how much our

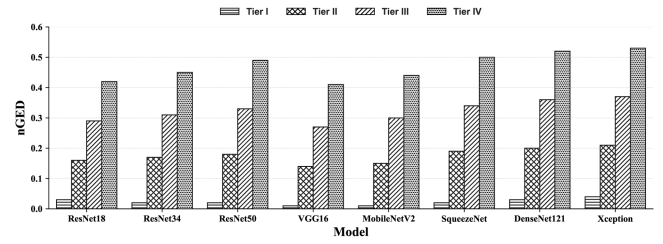


Fig. 7. Normalized Graph Edit Distance (nGED) between the original and obfuscated networks under Tiers I–IV. CRE in Tier IV drives the largest topological divergence.

obfuscation pipeline alters the topology of the original neural network's computation graph, independent of any specific attack scenario. Specifically, we measure the *graph-level* divergence between the original network G and the obfuscated network G' , highlighting the incremental impact of each MACO tier, especially the effects of CRE, which introduces additional dummy branches and at the TIR level.

We model each network as a directed acyclic graph (DAG), where nodes represent operators with attributes such as kernel size, channel dimension, etc. and edges represent dataflow connections. We then compute an approximate *graph edit distance* (GED) by enumerating the minimal set of node/edge insertions, deletions, or attribute modifications necessary to transform G into G' . To facilitate cross-model comparison, we have a normalized definition.

$$\text{nGED}(G, G') = \frac{\text{GED}(G, G')}{|V| + |E| + \sum_{\text{attr}} |A|}, \quad (13)$$

where $|V|$ and $|E|$ are the number of nodes and edges in the original graph, and $\sum_{\text{attr}} |A|$ is a simple count of all operator attributes under consideration.

We select eight types of models and compile each under four MACO tiers. For each tier, we compute the final obfuscated network G'_k and measure $\text{nGED}(G, G'_k)$. Here we focus purely on structural deviation and do not consider attacker strategies or the success/failure of a specific reverse-engineering approach.

Results: Fig. 7 plots the normalized GED values for the eight models under Tiers I–IV. As expected, Tier I barely changes the DAG structure, yielding minimal nGED (around 0.01–0.04). Tier II causes a more pronounced rise (to 0.14–0.21 in some cases) due to splitting (BS) and channel expansions (DP). Tier III further increases nGED, reflecting additional node insertions from CO/HA and kernel padding from KE. Finally, Tier IV shows the largest topological gap (up to around 0.50 or more) in certain architectures such as DenseNet121 and Xception, underscoring the disruptive effect of CRE's dummy branches and conditional paths.

2) *Impact on Attacks*: In this section, we investigate the extent to which MACO's multi-tier obfuscation framework hinders four representative attack methods, BTM [6], DND [5], DS [4], and REDLC [7]. Each of these attacks targets different dimensions of reverse-engineering, from symbolic execution and dynamic analysis to side-channel inference and DL-based extraction. We evaluate MACO's effectiveness using two metrics, OARA and OCA. We run each attack on eight commonly used networks and compare results for Tier I, Tier II,

TABLE I
TIER I ($\kappa = 0.3$, MAPO & CAP)

Network	BTD [6]		DND [5]		DS [4]		REDLC [7]	
	OARA	OCA	OARA	OCA	OARA	OCA	OARA	OCA
ResNet18	95.2%	100%	93.1%	100%	83.3%	100%	97.2%	100%
ResNet34	92.4%	100%	90.3%	100%	81.9%	100%	96.5%	100%
ResNet50	96.0%	100%	91.4%	100%	81.4%	100%	98.9%	100%
VGG16	97.3%	100%	-	-	86.5%	100%	97.3%	100%
MobileNetV2	93.9%	100%	91.8%	100%	82.7%	98.0%	99.0%	100%
SqueezeNet	92.3%	100%	-	-	73.1%	96.2%	96.2%	100%
DenseNet121	97.9%	100%	-	-	74.6%	99.3%	99.8%	100%
Xception	90.9%	100%	-	-	83.3%	98.5%	98.5%	100%

“-” means limited by conversion or analysis conditions.

TABLE II
TIER II RESULTS ($\kappa = 0.5$; DP $\alpha = 1.3$, BS $\beta = 0.4$)

Network	BTD [6]		DND [5]		DS [4]		REDLC [7]	
	OARA	OCA	OARA	OCA	OARA	OCA	OARA	OCA
ResNet18	76.4%	86.1%	70.8%	83.3%	51.4%	81.9%	80.6%	84.7%
ResNet34	71.5%	86.8%	67.4%	86.1%	50.7%	85.4%	80.0%	87.5%
ResNet50	76.8%	85.3%	65.5%	84.2%	47.5%	83.6%	83.6%	84.7%
VGG16	73.0%	83.8%	-	-	51.4%	78.4%	73.0%	81.1%
MobileNetV2	57.1%	70.4%	53.1%	67.3%	47.0%	66.3%	67.3%	69.4%
SqueezeNet	73.1%	84.6%	-	-	34.6%	80.8%	80.8%	88.5%
DenseNet121	81.6%	86.0%	-	-	35.0%	85.3%	91.6%	85.8%
Xception	62.1%	80.3%	-	-	34.7%	77.3%	77.3%	81.8%

“-” means limited by conversion or analysis conditions.

Tier III, and Tier IV of MACO. Below, we detail the parameters used for each tier and report the attacker’s OARA and OCA in the corresponding tables.

Tier I Setup and Results: Tier I includes only the low-overhead transformations MAPO, configured to reorder 20% of memory accesses randomly. CAP introduces a random alignment offset within a range of 16–64 bytes. We use a global parameter $\kappa = 0.3$ to indicate relatively conservative obfuscation. Table I shows the OARA and OCA for each of the four attacks across the eight networks. We observe that as MAPO and CAP mainly affect memory layout and cache alignment, they have only a moderate influence on BTD and DND’s attribute inferences, reducing OARA by around 2–10%. On the other hand, DS suffers a slightly larger drop (often up to 20%), because DS relies on monitoring memory access features. At this stage, REDLC shows virtually no drop since its extraction mechanism does not rely on signals such as memory alignment. Tier I does not modify the graph topology or operator connections, so all four attacks still achieve relatively high connection recovery.

Tier II Setup and Results: Tier II includes MAPO and CAP (same as Tier I), and DP with expansion factor $\alpha = 1.3$, BS with partition ratio $\beta = 0.4$, SD injecting redundant add-sub operations. We set the global parameter $\kappa = 0.5$ to reflect more extensive modifications than Tier I. These newly introduced transformations alter operator shapes (DP), split or fuse dataflows (BS), and insert small no-op branches (SD), thus potentially confusing attribute inference across a broader range of operators. The results are shown in Table II. Compared with Tier I, all four attacks, including REDLC, now experience a noticeable drop in OARA, as DP and BS complicate the identification of true operator parameters. BTD and DND degrade by around 15–30% (depending on network complexity), while DS drops to the 35–50% range. REDLC also declines, typically to 75–90%, indicating that the newly introduced structural modifications hamper purely DL-based reconstruction as well. The obfuscated

TABLE III
TIER III RESULTS ($\kappa = 0.8$; DP $\alpha = 1.5$, KE WITH $\delta_1 = \delta_2 = 2$)

Network	BTD		DND		DS		REDLC	
	OARA	OCA	OARA	OCA	OARA	OCA	OARA	OCA
ResNet18	60.5%	45.8%	55.6%	31.9%	26.4%	27.8%	69.4%	41.7%
ResNet34	56.3%	59.7%	50.0%	43.8%	25.0%	53.5%	47.2%	63.9%
ResNet50	38.4%	65.0%	29.4%	37.3%	19.8%	28.2%	37.9%	67.2%
VGG16	45.9%	28.6%	-	-	27.0%	13.0%	48.6%	45.9%
MobileNetV2	23.5%	36.7%	16.3%	23.5%	27.3%	20.4%	32.7%	33.7%
SqueezeNet	53.8%	30.8%	-	-	15.4%	42.3%	46.2%	50.0%
DenseNet121	32.4%	65.3%	-	-	14.0%	51.3%	30.1%	67.6%
Xception	36.4%	57.6%	-	-	9.2%	33.3%	39.4%	66.6%

“-” means limited by conversion or analysis conditions.

TABLE IV
TIER IV RESULTS ($\kappa = 1.0$, $\eta = 0.6$, WITH ALL TIER III TRANSFORMATIONS PLUS CRE)

Network	BTD		DND		DS		REDLC	
	OARA	OCA	OARA	OCA	OARA	OCA	OARA	OCA
ResNet18	16.7%	5.6%	11.1%	1.4%	4.2%	2.8%	18.1%	6.9%
ResNet34	29.2%	7.6%	23.6%	6.3%	4.2%	6.9%	31.3%	11.1%
ResNet50	27.1%	8.2%	25.4%	6.8%	11.9%	2.8%	29.4%	11.3%
VGG16	24.3%	5.4%	-	-	16.2%	2.7%	27.0%	8.1%
MobileNetV2	21.4%	6.1%	15.3%	1.0%	12.1%	2.0%	18.4%	3.1%
SqueezeNet	26.9%	7.7%	-	-	7.7%	3.8%	23.1%	7.7%
DenseNet121	15.6%	4.4%	-	-	8.2%	2.3%	17.5%	3.5%
Xception	27.3%	7.6%	-	-	5.1%	3.0%	24.2%	6.1%

“-” means limited by conversion or analysis conditions.

operators’ edges also experience a 15%–20% drop in OCA, yet the adversary can still recover the majority of connections at this stage. These OCA values indicate that, although the nodes themselves appear altered or split, the core network connectivity remains partially identifiable.

Tier III Setup and Results: Building on Tier II, Tier III enables CO, HA, KE. In addition to DP, BS, and SD from Tier II, we keep $\kappa = 0.8$, intensifying the expansions and the probability of injecting structural changes. Specifically, DP uses $\alpha = 1.5$ now, and BS maintains $\beta = 0.4$. MAPO and CAP remain active as well. Table III presents the Tier III results. Relative to Tier II, OARA drops slightly further (by an additional 50% on average), whereas OCA now decreases more noticeably. Specifically, the extra dummy arithmetic in CO and the deeper structure from HA marginally reduce each attacker’s ability to deduce correct operator dimensions. KE contributes by disguising kernel shapes with large zero-padded areas. As a result, typical OARA values for DS, for example, dip below 15% on more complex networks such as DenseNet121 or Xception. On the other hand, with HA adding new identity layers and SD continuing to inject branches, many attacks experience confusion in identifying true layer-to-layer connectivity. OCA can fall to around 33.3% or even lower for particularly branched architectures like DenseNet121 and Xception.

Tier IV Setup and Results: Tier IV incorporates all transformations from Tiers I–III and additionally enables CRE. We fix $\kappa = 1.0$. And a separate global parameter $\eta = 0.6$ controlling the fraction of eligible operators where CRE dummy branches are injected. During TIR lowering, CRE introduces conditional or random-index dummy paths, which are numerically neutral but add considerable extraneous computations and unplanned memory accesses. As shown in Table IV, Tier IV drives OARA down by another 50% compared to Tier III, and the changes in OCA likewise reflect the disruptive impact of CRE. Empirically,

TABLE V
CPU PERFORMANCE: INFERENCE LATENCY (MS) AND MEMORY OVERHEAD (%)

Network	Vanilla		Tier I		Tier II		Tier III		Tier IV	
	Latency	Overhead	Latency	Overhead	Latency	Overhead	Latency	Overhead	Latency	Overhead
ResNet18	59	0%	63	+4.2%	70	+9.6%	89	+15.3%	107	+21.7%
ResNet34	118	0%	124	+4.7%	148	+9.9%	185	+16.2%	215	+23.4%
ResNet50	125	0%	137	+4.4%	168	+10.1%	203	+15.8%	230	+24.3%
VGG16	510	0%	554	+3.9%	601	+8.5%	851	+14.6%	943	+21.8%
MobileNetV2	9	0%	10	+4.1%	12	+9.2%	14	+15.5%	17	+25.1%
SqueezeNet	26	0%	29	+4.3%	32	+9.7%	41	+13.8%	48	+19.2%
DenseNet121	95	0%	105	+5.1%	125	+10.9%	157	+16.4%	182	+22.7%
Xception	276	0%	307	+4.8%	360	+9.8%	454	+15.9%	524	+21.5%

Each column pair shows Latency (left) and Overhead (right).

we observe that OARA can drop below 30%, and in certain cases even fall under 5%. For the DS attack on Xception, the OARA stands at 5.1%, marking an 85.8% reduction compared to its original value of 90.9% prior to obfuscation. The OCA results are similarly encouraging, the random control flow illusions created by CRE add spurious branches or merged paths, causing attackers to misidentify which sub-branches are part of the ‘real’ computation. Consequently, for most representative networks, the final OCA often remains below 10%, in the best-case scenario, the obfuscation rate reaches an impressive 99%.

In summary, Tier IV yields the highest level of confusion for both attributes and connectivity. Although OARA sees only incremental changes over Tier III, OCA suffers notably larger reductions due to the conditional dummy branches introduced by CRE. Across Tiers I to IV, MACO progressively degrades attackers’ ability to OARA and OCA. Tier I primarily disrupts side-channel-based inferences of operator parameters (most notably affecting DS), while Tiers II and III introduce fundamental shape and structural obfuscations that impact all four attacks. Finally, Tier IV adds dynamic dummy branching (CRE), further confusing connectivity analysis.

We emphasize that even at Tier IV, the functional correctness of the model remains unaltered. The overhead of each tier, along with a deeper analysis of performance trade-offs, is reported in Section VI-D.

D. Performance Overhead of MACO

Having established MACO’s efficacy in obfuscating model structures, we evaluate the performance overhead it introduces at different tiers of obfuscation. In particular, we measure two metrics on both CPU and GPU platforms.

- *Latency (ms)*: The time required to run a single inference (batch size = 1) after compilation with TVM at optimization level -O0. This reflects the raw execution cost.
- *Memory Overhead (%)*: The relative increase in peak memory usage compared to an unobfuscated (Vanilla) baseline. We measure the highest runtime memory consumption during inference by system-level monitoring. The overhead is reported as $\frac{\text{Mem}_{\text{obt}} - \text{Mem}_{\text{vanilla}}}{\text{Mem}_{\text{vanilla}}} \times 100\%$.

We run single-image inferences on the same eight networks from our prior evaluation under five configurations. Vanilla TVM (no obfuscation), and tiers I–IV of MACO. We compile

each model using TVM at level-O0 to avoid aggressive optimizations that might mask or eliminate obfuscation effects. For each model and tier, we record the inference latency in seconds and the memory overhead relative to the vanilla baseline.

CPU Results: Table V presents the latency and memory overhead on the CPU. As shown, Tier I imposes a modest time penalty (roughly +10% on average) and a small memory overhead (under +5% in most cases). With Tier II, execution times can increase by 30-40% for deeper models (ResNet50 or DenseNet121), while memory overhead climbs toward +10%. Tier III sees a 50-70% time overhead and up to +15-20% memory overhead, primarily due to the insertion of identity layers and kernel expansions. Finally, Tier IV has the highest overhead, reaching 80-90% above baseline, and memory usage can jump by 20-25% (occasionally more in particularly heavy architectures like DenseNet121).

GPU Results: Table VI shows the same eight models running on an NVIDIA RTX 3090 GPU. As expected, the overhead trends are similar. The introduction of each tier progressively increases runtime, with Tier IV again nearly doubling the latency in some cases. Memory overhead also follows a similar pattern, though the absolute GPU overhead percentages can be slightly larger for more complex networks, where kernel expansions (DP/KE) and extra identity layers (HA) cause multiple reconfigurations of GPU memory buffers.

Overall, Tier I is lightweight, adding minimal disruptions at the memory and scheduling levels. Tiers II-IV progressively increase overheads. Despite these overheads, we consider the trade-off justifiable in scenarios that require robust anti-extraction protection, especially for high-value models deployed at the edge or in untrusted environments.

E. Output Consistency

Having analyzed MACO’s obfuscation efficacy and its associated overhead, we next address the question of accuracy preservation. Specifically, we evaluate whether the obfuscated networks produced by MACO yield inference outputs that are semantically and numerically consistent with their original counterparts. To this end, we compile each of our eight models under all four obfuscation tiers in TVM, and conduct inference on the same controlled test inputs for both the obfuscated and vanilla versions, without any retraining or parameter modification.

TABLE VI
GPU PERFORMANCE: INFERENCE LATENCY (MS) AND MEMORY OVERHEAD (%)

Network	Vanilla		Tier I		Tier II		Tier III		Tier IV	
	Latency	Overhead	Latency	Overhead	Latency	Overhead	Latency	Overhead	Latency	Overhead
ResNet18	5.0	0%	5.3	+3.6%	6.1	+7.5%	6.6	+12.4%	7.2	+19.6%
ResNet34	11.0	0%	11.9	+4.2%	12.7	+9.8%	14.1	+14.6%	15.4	+22.2%
ResNet50	14.0	0%	14.5	+3.8%	15.2	+8.9%	16.0	+14.0%	17.0	+21.4%
VGG16	52.0	0%	57.1	+4.1%	62.8	+9.3%	69.1	+13.7%	72.5	+20.1%
MobileNetV2	3.0	0%	3.6	+3.7%	3.9	+8.4%	4.1	+13.2%	4.6	+19.8%
SqueezeNet	10.0	0%	10.8	+4.5%	11.9	+9.1%	12.5	+14.7%	13.0	+20.3%
DenseNet121	32.0	0%	36.2	+3.9%	39.8	+8.7%	46.8	+14.9%	49.1	+22.5%
Xception	42.0	0%	43.8	+4.2%	46.0	+9.5%	48.3	+15.1%	51.2	+21.9%

Each column pair shows Latency (left) and Overhead (right).

TABLE VII
OUTPUT CONSISTENCY METRICS UNDER MACO OBFUSCATION

Obf. Tier	Top-1 Label Match (%)	Max Abs Deviation	Confidence Deviation
Tier I	100%	$< 10^{-7}$	$< 10^{-6}$
Tier II	100%	$< 10^{-7}$	$< 10^{-6}$
Tier III	100%	$< 10^{-6}$	$< 10^{-5}$
Tier IV	100%	$< 10^{-6}$	$< 10^{-5}$

All evaluated networks show consistent semantic outputs.

To comprehensively assess consistency, we adopt three complementary metrics. First, we measure the top-1 label match rate, defined as the percentage of test cases where the top-1 classification result from the obfuscated model matches that of the original model. This directly reflects semantic equivalence at the output level. Second, we compute the maximum absolute deviation between the output tensors of the original and obfuscated models (prior to softmax), which captures the largest element-wise numeric deviation introduced by obfuscation. Third, we evaluate prediction confidence deviation, defined as the maximum absolute difference in softmax output probabilities across all classes. This metric detects any perturbation to the predicted confidence scores, even when the top-1 class remains unchanged.

Our experiments use a mix of real and randomly generated inputs that cover representative tensor shapes and value ranges. All metrics are aggregated over 500 inference trials per model per tier. The results, summarized in Table VII, demonstrate that MACO preserves 100% top-1 accuracy across all models and tiers. Moreover, the observed numeric deviations remain within machine-precision bounds, less than 10^{-6} for absolute error and less than 10^{-5} for confidence deviation, which are in line with accepted thresholds in compiler-level correctness testing. These results affirm MACO's design principle while bitwise identity is not guaranteed due to dynamic branching and floating-point reordering, the essential computation semantics and prediction behavior remain unaffected by obfuscation.

F. Comparison

In our final research question, we provide a qualitative comparison of MACO against two notable compiler-based defense strategies from the literature, NNReArch [9] and SecureTVM [8]. Since both approaches remain closed-source and

TABLE VIII
COMPARISON WITH THE SOTA

	NNReArch	SecureTVM	Maco
Hierarchical Obfuscation	×	×	✓
Memory Access Perturbation	✓	×	✓
Operator Attributes Expansion	×	×	✓
Dummy Branching	×	×	✓
Side Channel-based Attack	✓	×	✓
Learning-based Attack	×	×	✓
Symbolic Execution-based Attack	×	×	✓
Architectural-Level Protection	×	×	✓
Execution Environment Dependency	Weak	Strong	Weak
Heavy Computation	Medium	High	Low(Controllable)

‘✓’ and ‘×’ represent the presence and absence of specific characteristics in various studies, respectively.

cannot be directly benchmarked with our pipeline, we focus on the high-level design features they implement, contrasting them with MACO's multi-level obfuscation methods.

As summarized in Table VIII, NNReArch primarily investigates the correlation between low-level tensor programs and signals extracted from side-channel attacks. However, due to its lack of protection for operator connection edges and attributes, it remains partially vulnerable to side-channel or memory-trace-based detection, which can still reveal structural hints. SecureTVM focuses on secure parameter processing, often leveraging trusted execution environments. While this ensures strong confidentiality for model parameters, it does not obfuscate operator attributes or connection paths in a way that effectively prevents architecture reverse engineering. Although SecureTVM is integrated at the TVM level, its primary focus is data confidentiality rather than operator or topology obfuscation, and it incurs notable runtime overhead.

In contrast, MACO offers a diverse set of obfuscation techniques, including attribute expansions, memory-level perturbations, and dummy branching. This comprehensive approach strengthens resilience against static, dynamic, and side-channel-based reverse engineering attempts, while allowing customizable overhead control.

VII. DISCUSSION

A. Framework Level or Compiler Stack Level?

While certain obfuscation transformations, such as injecting dummy branches or applying mathematically equivalent rewrites, can in theory be scripted at the framework level

(e.g., in PyTorch), implementing them at the compiler level offers stronger guarantees and broader coverage. Compiler-based obfuscation operates on the intermediate representation (IR), enabling unified transformations across all operators. It ensures that obfuscation is preserved through lowering and code generation, rather than being discarded or optimized away by downstream passes. Moreover, only at the compiler level can transformations precisely target memory layout, scheduling behavior, and low-level control flow-dimensions inaccessible from high-level modeling frameworks. This integration makes MACO robust against both static and dynamic extraction, and avoids entangling obfuscation logic with model training or upstream toolchains.

B. Limitations

Compilation Pipeline Constraints: MACO's passes are tightly interwoven with TVM's Relay and TIR structures, porting these passes to other compilers or frameworks would require re-engineering the transformations. While many principles would carry over, the implementation details heavily depend on TVM's pass infrastructure and scheduling APIs. That said, most core transformations in MACO are conceptually backend-agnostic and could be re-implemented within other compiler frameworks that expose comparable intermediate representations. Achieving full portability would require adapting to each compiler's transformation granularity, pass orchestration, and backend-specific optimization pipeline, which we leave as promising future work.

Dynamic Compilation Support: MACO currently assumes that the network's computation graph is known at compile time (as is typical in TVM's ahead-of-time compilation). For highly dynamic models or runtime shape changes, additional logic would be needed to keep obfuscation effective without causing functional errors. This limitation is particularly relevant when considering the applicability of MACO to non-convolutional architectures such as Transformers and large language models, where input-dependent control flow, dynamic sequence lengths, and attention-based token interactions pose new challenges. Adapting MACO to support such models would necessitate integration with dynamic shape-aware passes and backend-specific scheduling schemes, which we leave for future exploration.

Functional versus Bitwise Equivalence: MACO is designed to maintain functional (mathematical) equivalence, not strict bitwise identity. Transformations like expression rewriting or memory reordering can alter the sequence of floating-point operations. Consequently, outputs may differ at the bit level due to floating-point non-associativity, a well-accepted characteristic in modern compilers (e.g., TVM, LLVM) that do not ensure bitwise determinism. Our validation therefore focuses on preserving semantic correctness and numerical stability, aligning with our goal of semantic obfuscation over deterministic reproducibility.

C. Potential Extensions

Scheduling Integration: Future work can explore synergy between MACO's obfuscation passes and advanced

auto-schedulers. For instance, specialized cost models might preserve obfuscation transformations while still achieving near-optimal performance.

Integration of Hardware-Specific Features: On specialized hardware backends (e.g., NPU, ASICs), new obfuscation techniques, such as rearranging on-chip buffer usage, might extend MACO's approach beyond CPU/GPU architectures.

VIII. CONCLUSION

MACO presents a novel multi-tier approach to compiler-level obfuscation for DNN, focusing on topology and execution trace confusion. It is complementary to encryption or enclave-based methods that protect the model weights. By offering tunable levels of obfuscation (Tiers I-IV), MACO covers a wide range of deployment scenarios, from lightweight memory reordering to comprehensive, branching-heavy transformations. The experimental results show that MACO achieves strong resilience against reverse-engineering attacks without compromising the model's functional correctness.

REFERENCES

- [1] T. Chen et al., "TVM: An automated end-to-end optimizing compiler for deep learning," in *Proc. 13th USENIX Symp. Operating Syst. Des. Implementation*, 2018, pp. 578–594.
- [2] Y. Hu et al., "Privacy risks of federated knowledge graph embedding: New membership inference attacks and personalized differential privacy defense," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 3, pp. 2788–2805, May/Jun. 2025.
- [3] X. Gong et al., "Augmenting model extraction attacks against disruption-based defenses," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 531–546, 2025.
- [4] X. Hu et al., "DeepSniffer: A DNN model extraction framework based on learning architectural hints," in *Proc. 25th Int. Conf. Architect. Support Program. Lang. Operating Syst.*, 2020, pp. 385–399.
- [5] R. Wu, T. Kim, D. J. Tian, A. Bianchi, and D. Xu, "DnD: A cross-architecture deep neural network decompiler," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 2135–2152.
- [6] Z. Liu, Y. Yuan, S. Wang, X. Xie, and L. Ma, "Decompiling x86 deep neural network executables," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 7357–7374.
- [7] M. Li et al., "REDLC: Learning-driven reverse engineering for deep learning compilers," in *Proc. IEEE 35th Int. Symp. Softw. Rel. Eng.*, 2024, pp. 204–215.
- [8] P.-H. Huang et al., "SecureTVM: A TVM-based compiler framework for selective privacy-preserving neural inference," *ACM Trans. Des. Automat. Electron. Syst.*, vol. 28, no. 4, pp. 1–28, 2023.
- [9] Y. Luo, S. Duan, C. Gongye, Y. Fei, and X. Xu, "NNReArch: A tensor program scheduling framework against neural network architecture reverse engineering," in *Proc. IEEE 30th Annu. Int. Symp. Field-Programmable Custom Comput. Mach.*, 2022, pp. 1–9.
- [10] XLA: Accelerated Linear Algebra, (n.d.). [Online]. Available: <https://github.com/openxla/xla>
- [11] C. Lattner et al., "MLIR: Scaling compiler infrastructure for domain specific computation," in *Proc. 2021 IEEE/ACM Int. Symp. Code Gener. Optim.*, 2021, pp. 2–14.
- [12] X. Wu, P. Paramasivam, and V. Taylor, "Autotuning apache TVM-based scientific applications using Bayesian optimization," in *Proc. Workshops Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2023, pp. 29–35.
- [13] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe, "Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines," *ACM SIGPLAN Notices*, vol. 48, no. 6, pp. 519–530, 2013.
- [14] Q. Pan, J. Wu, X. Lin, and J. Li, "Side-channel analysis-based model extraction on intelligent CPS: An information theory perspective," in *Proc. 2021 IEEE Int. Conf. Internet of Things IEEE Green Comput. Commun. IEEE Cyber Phys. Social Comput. IEEE Smart Data IEEE Congr. Cybern.*, 2021, pp. 254–261.

- [15] Z. Fang et al., "Zero-query adversarial attack on black-box automatic speech recognition systems," in *Proc. 2024 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2024, pp. 630–644.
- [16] J. Wei, Y. Zhang, Z. Zhou, Z. Li, and M. A. Al Faruque, "Leaky DNN: Stealing deep-learning model secret with GPU context-switching side-channel," in *Proc. 50th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2020, pp. 125–137.
- [17] H. Naghibijouybari, A. Neupane, Z. Qian, and N. Abu-Ghazaleh, "Rendered insecure: GPU side channel attacks are practical," in *Proc. 2018 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 2139–2153.
- [18] Z. Yang et al., "Defending data inference attacks against machine learning models by mitigating prediction distinguishability," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 3, pp. 2687–2704, May/Jun. 2025.
- [19] B. Olney, "Secure reconfigurable computing paradigms for the next generation of artificial intelligence and machine learning applications," PhD dissertation, Dept. Comput. Sci. Eng., University of South Florida, Tampa, FL, USA, 2023.
- [20] R. Wang et al., "Free fine-tuning: A plug-and-play watermarking scheme for deep neural networks," in *Proc. 31st ACM Int. Conf. Multimedia*, 2023, pp. 8463–8474.
- [21] J. M. Vidal and M. A. Sotelo Monge, "Obfuscation of malicious behaviors for thwarting masquerade detection systems based on locality features," *Sensors*, vol. 20, no. 7, 2020, Art. no. 2084.
- [22] Z. Huang, W.-J. Lu, C. Hong, and J. Ding, "Cheetah: Lean and fast secure two-party deep neural network inference," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 809–826.
- [23] ONNX model Zoo, (n.d.). [Online]. Available: <https://github.com/onnx/models>
- [24] K. Simonyan, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.
- [25] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [26] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 4700–4708.
- [27] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 1251–1258.
- [28] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 4510–4520.
- [29] F. N. Iandola, "SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and < 0.5 MB model size," 2016, *arXiv:1602.07360*.



Yang Li is currently working toward the PhD degree with the Nanjing University of Aeronautics and Astronautics, China. His research interests include deep learning model security, deep learning compiler, and model deployment optimization.



Minghui Li is currently working toward the PhD degree with the Nanjing University of Aeronautics and Astronautics, China. Her research interests include deep learning security, deep learning compiler, and information security.



Hao Han (Member, IEEE) received the BS degree in computer science and technology from Nanjing University, Nanjing, China, in 2005, and the PhD degree in computer science from the College of William and Mary, Williamsburg, VA, USA, in 2014. He is currently a professor with the College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, China. His research interests include system, and software security against various types of threats.



Liming Fang (Member, IEEE) received the PhD degree in computer science from the Nanjing University of Aeronautics and Astronautics, in 2012. He is currently a professor with the College of Computer Science, Nanjing University of Aeronautics and Astronautics. He has published more than 100 papers in prestigious journals and conferences. His current research interests include information security, applied cryptography, AI security, blockchain, proxy re-encryption, public-key encryption with keyword search, and anonymous systems. He was an organizing chair for CryptoIC 2019 and Inscrypt 2019, and the secretary-general of SpaCCS 2020.

ing chair for CryptoIC 2019 and Inscrypt 2019, and the secretary-general of SpaCCS 2020.